NAME:-Shubham Suryakant Biradar
ROLL NO.:-A-12
BATCH:-A1
ASSINGMENT:-1

Aim:- Use sequence data types and their associated operations in Python a) Strings b) Lists c) Arrays d) Tuples e) Dictionary f) Sets g) Range

THEORY:-Q.1.WhataresequencedatatypesinPython?Namethemandexplainhowindexing and slicing work across different sequence type?
** Answer:**- **Sequence data types in Python** are data types that store multiple items in an ordered manner. Each item has a position number called an **index**.

Main Sequence Data Types in Python:

1. String(`str`) – Sequence of characters. Example: `"Hello"`
2. List (`list`) – Sequence of different data types, mutable. Example: `[1, 2, 3]`
3. Tuple(`tuple`) – Ordered sequence, immutable. Example: `(10,20, 30)`
4. Range(`range`) – Sequence of numbers. Example: `range(5)`
5. Bytes(`bytes`) – Immutable sequence of bytes.
6. Bytearray(`bytearray`) – Mutable sequence of bytes.

Indexing in Sequence Types:

Indexing is used to access elements by position. Index starts from **0**. Negative indexing starts from **-1** (last element).

Example with list:

- `a = [10, 20,30,40]`
- `a[0]` → `10`
- `a[2]` → `30`
- `a[-1]` → `40`

This works similarly for strings, tuples, and other sequence types.

Slicing in Sequence Types:

Slicing is used to access a part of the sequence.
Syntax: `sequence[start : stop : step]`

Example with string:

- `s = "Python"`
- `s[0:3]` → `"Pyt"`
- `s[2:5]` → `"tho"`
- `s[::-1]` → `"nohtyP"`

Example with tuple:

- `t = (1,2 ,3, 4, 5)`
- `t[1:4]` → `(2, 3, 4)`

Common Behavior Across Sequence Types:

- Support **indexing** and **slicing**.
- Maintain order of elements.
- Can use `len()` to find length.
- Can iterate using loops.

Q.2. What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications? **Answer:- Difference between List and Tuple in Python:**

Feature	List	Tuple
Syntax	`[ ]`	`( )`
Mutability	Mutable (can change)	Immutable (cannot change)
Performance	Slower	Faster
Memory Usage	More memory	Less memory
Methods	Many methods (`append()`, `remove()`)	Few methods (`count()`, `index()`)

Example:

- List: `a = [1, 2, 3]`
- Tuple: `b = (1, 2, 3)`

Why Prefer Tuple Over List in Real-World Applications?

1. **Data Safety** – Tuple cannot be changed, so important data stays fixed. Example: Days of week, months, coordinates.
2. **Faster Performance** – Tuples are faster than lists for reading data.
3. **Less Memory Usage** – Useful when handling large fixed datasets.
4. **Used as Dictionary Keys** – Tuples can be used as keys in dictionaries, lists cannot.
5. **Fixed Records** – Good for storing records like (ID, Name, Age).

Example Uses:

- GPS coordinates: `(19.0760, 72.8777)`
- RGB color values: `(255, 0, 0)`
- Database records

Q.3. How are Python arrays different from lists? When should you use the array module instead of a list? Answer:- Difference between Python Arrays and Lists:

Feature	Array	List
Data Type	Stores same type elements only	Stores different data types
Module Required	Need array module	Built-in type
Memory Usage	Less memory efficient	More memory usage
Speed	Faster for numeric operations	Slower for large numeric data
Flexibility	Less flexible	More flexible

Example:

```
from array import array
a = array('i', [1, 2, 3])   # integer array

b = [1, 2, 3, "Hello"]      # list
```

When to Use `array` Module Instead of List?

1. **When storing only numeric data** Example: integers, floats.

2. **When memory efficiency is important** Arrays use less memory than lists.
3. **When working with large datasets** Faster for many numeric values.
4. **When type consistency is needed** All elements must be same type.

When to Use List?

- When storing mixed data types.
- When more flexibility is needed.
- For general-purpose programming.

Q.4.Explainhow dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples?
Answer:- Dictionaries in Python store data in **key-value pairs** such as:

```
student = {"name": "Lokesh", "age": 20}
```

Here, `"name"` and `"age"` are keys, while `"Lokesh"` and `20` are values.

How Dictionaries Work Internally:

1. Hash Table Structure

Python dictionaries use a **hash table** internally. A hash table stores keys and values in memory locations for quick access.

2. Hashing the Key

When a key is added, Python uses the `hash()` function on the key.

```
hash("name")
```

This creates a numeric hashvalue.

3. Finding Storage Location

Then Python decide where to store the key-value pair.

4. Lookup Process

When you write:

```
student["name"]
```

5. Collision Handling

`"name"` again and directly jumps to the correct location to get the value.

Why Dictionary Lookups Are Faster Than Lists or Tuples:

Dictionary:

- Uses hash table.
- Directly finds location using key.
- Average lookup time = **O(1)** (constant time).

List / Tuple:

- Use ordered indexing.
- If searching by value, Python checks items one by one.
- Lookup time = **O(n)** (linear time).

Example:

```
nums = [10, 20, 30, 40]
30 in nums
```

Python may scan elements one by one.

```
data = {"a": 10, "b": 20}
data["b"]
```

Python directly finds `"b"` quickly.

Q5 What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Answer:- **Difference between Sets and Lists in Python:**

Feature	List	Set
Syntax	`[ ]`	`{ }`
Order	Ordered	Unordered
Duplicate Values	Allowed	Not allowed
Indexing	Supporte	Not supported
Mutability	d Mutable	Mutable (set)

Example:

```
a = [1, 2, 2, 3]
b = {1, 2, 2, 3}
```

Output:

- List → `[1, 2, 2, 3]`
- Set → `{1, 2, 3}`

How Sets Handle Duplicate Values:

Sets automatically remove duplicate values. When duplicate elements are added, only one copy is stored.

Example:

```
s = {10, 20, 20, 30}
print(s)
```

Output:

```
{10, 20, 30}
```

Where Sets Are Useful in Practical Scenarios:

1. **Removing Duplicates** From a list of repeated values.
2. **Membership Testing** Checking if an item exists is fast.
3. **Mathematical Operations** Union, intersection, difference.

4. **Unique Usernames / IDs** Store only unique values.
5. **Tags / Categories** Avoid repeated entries.

Example:

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)   # Union
print(A & B)   # Intersection
```

```
#CODE:-

# a) Strings
# 1. Create a string and display its length and first three characters
s = "PythonProgramming"
print("String:", s)
print("Length:", len(s))
print("First three characters:", s[:3])
```

```
String: PythonProgramming
Length: 17
First three characters: Pyt
```

```
#b) Lists
# 2. Create a list of 5 integers and print max and min values
lst = [10, 25, 5, 40, 15]
print("\nList:", lst)
print("Maximum value:", max(lst))
print("Minimum value:", min(lst))
```

```
List: [10, 25, 5, 40, 15]
Maximum value: 40
Minimum value: 5
```

```
# d) Tuples
# 3. Create tuple of student names and display using loop
students = ("Lokesh", "Rahul", "Amit", "Sneha")
print("\nTuple Elements:")
for name in students:
    print(name)
```

```
Tuple Elements:
Lokesh
Rahul
Amit
Sneha
```

```
# e) Dictionary
# 4. Create dictionary with roll number and name, print all keys
student_dict = {101: "Lokesh", 102: "Rahul", 103: "Amit"}
print("\nDictionary:", student_dict)
print("Keys:")
for key in student_dict.keys():
    print(key)
```

```
Dictionary: {101: 'Lokesh', 102: 'Rahul', 103: 'Amit'}
Keys:
101
```

```
102
103
```

```
# f) Sets
# 5. Create set of numbers and remove duplicates from list
num_set = {1, 2, 3, 4, 5}
print("\nSet:", num_set)

dup_list = [1, 2, 2, 3, 4, 4, 5]
unique_set = set(dup_list)
print("List after removing duplicates:", unique_set)
```

```
Set: {1, 2, 3, 4, 5}
List after removing duplicates: {1, 2, 3, 4, 5}
```

```
# 6. Count vowels in a given string
vowels = "aeiouAEIOU"
count = 0
for ch in s:
    if ch in vowels:
        count += 1
print("Vowel count:", count)
```

```
Vowel count: 4
```

```
# 7. Insertion and deletion operations on list
lst.append(50)          # Insert at end
lst.insert(2, 100)     # Insert at index 2
print("After insertion:", lst)

lst.remove(25)         # Delete value
lst.pop()             # Delete last element
print("After deletion:", lst)
```

```
After insertion: [10, 25, 100, 5, 40, 15, 50]
After deletion: [10, 100, 5, 40, 15]
```

```
# 8. Convert list into tuple and tuple into list
tuple_from_list = tuple(lst)
print("List to Tuple:", tuple_from_list)

list_from_tuple = list(students)
print("Tuple to List:", list_from_tuple)
```

```
List to Tuple: (10, 100, 5, 40, 15)
Tuple to List: ['Lokesh', 'Rahul', 'Amit', 'Sneha']
```

```
# c) Arrays
from array import array

arr = array('i', [1, 2, 3, 4, 5])
print("\nArray:", arr)

arr.append(6)
print("After append:", arr)

arr.remove(3)
print("After remove:", arr)
```

```
Array: array('i', [1, 2, 3, 4, 5])
After append: array('i', [1, 2, 3, 4, 5, 6])
After remove: array('i', [1, 2, 4, 5, 6])
```

```
# g) Range
r = range(1, 11)
print("\nRange values:")
for i in r:
    print(i, end=" ")
```

```
Range values:
1 2 3 4 5 6 7 8 9 10
```

CONCLUSION:- In this practical, we studied and used different sequence data types in Python such as strings, lists, arrays, tuples, dictionaries, sets, and range. Each data type has its own features and uses in programming.

We learned how to perform various operations like indexing, slicing, insertion, deletion, searching, and traversal. Strings helped in handling text data, lists and arrays were useful for storing and modifying collections of values, tuples ensured data safety with immutability, dictionaries allowed storing data in key-value pairs, sets helped in removing duplicates and performing mathematical operations, and range was useful for generating sequences of numbers.

Overall, this experiment helped us understand how to efficiently store, access, and manipulate data using different Python data types, which is essential for writing effective and optimized programs.

I prefer this response